Computer Programming Languages:

- A **programming language** is an artificial language that can be used to control the behavior of a machine, particularly a computer

- Programming languages, like human languages, are defined through the use of syntactic and semantic rules, to determine structure and meaning respectively.

Computer Programming Languages (Contd…):

- Programming languages are used to facilitate communication about the task of organizing and manipulating information, and to express algorithms precisely.

- For 50 years, computer programmers have been writing code. New technologies continue to emerge, develop, and mature at a rapid pace. Now there are more than 2,500 documented programming languages!

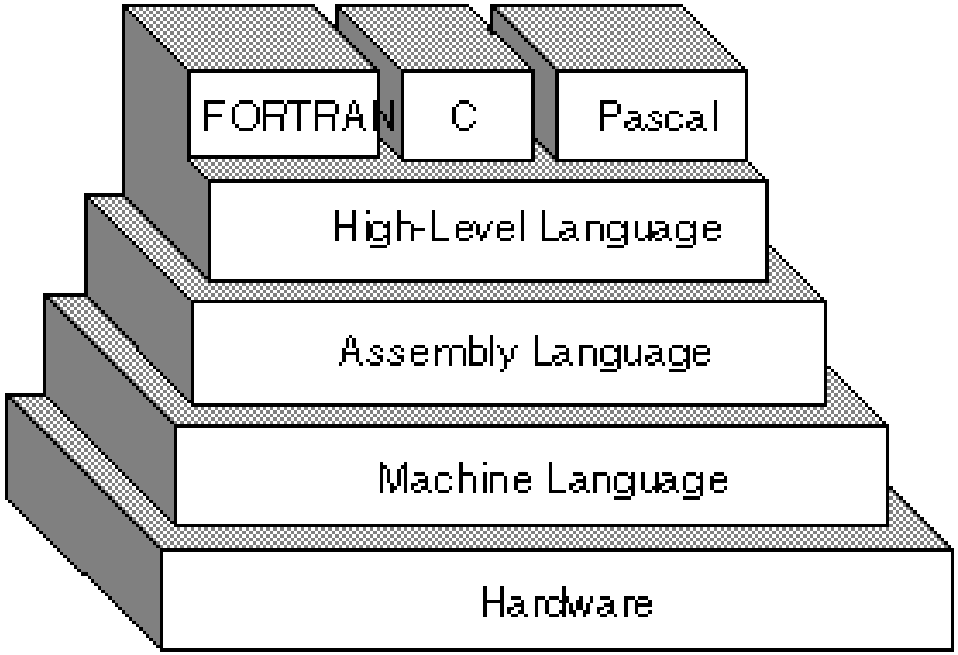
Non-computational languages:

- **Non-computational languages**, such as markup languages like HTML are usually not considered programming languages.

- Often a programming language is embedded in the non-computational language.

Machine language:

- It is the lowest-level programming language.



- Machine languages are the only languages understood by computers.

Machine language:

- While easily understood by computers, machine languages are almost impossible for humans to use because they consist entirely of numbers.

For example, a processor can execute the following binary instruction as expressed in machine language:

```
Binary: 10110000 01100001 (Hexadecimal: 0xb061)
```

Assembly Level Language:

- An **assembly language** is a low-level language for programming computers.

- The word "**low**" does not imply that the language is inferior to high-level programming languages but rather refers to the small or nonexistent amount of abstraction between the language and machine language, because of this, low-level languages are sometimes described as being "**close to the hardware**."

- It implements a symbolic representation of the numeric machine codes and other constants needed to program a particular CPU architecture.

Assembly Level Language (contd…):

- A utility program called an **assembler**, is used to translate assembly language statements into the target computer's machine code.
- The assembler performs a more or less isomorphic translation (a one-to-one mapping) from mnemonic statements into machine instructions and data.

Example: Assembly language representation is easier to remember (more *mnemonic*)

mov al, 061h

This instruction means:

Move the hexadecimal value 61 (97 decimal) into the processor register named "al".

The mnemonic "mov" is an *operation code* or *opcode*, A comma-separated list of arguments or parameters follows the opcode;

Example (Adds 2 numbers):

```
name "add"
mov al, 5          ; bin=00000101b
mov bl, 10         ; hex=0ah or bin=00001010b
add bl, al         ; 5 + 10 = 15 (decimal) or hex=0fh or
                   bin=00001111b
```

High-level language:

- **High-level languages** are relatively easy to learn because the instructions bear a close resemblance to everyday language, and because the programmer does not require a detailed knowledge of the internal workings of the computer.

- Each instruction in a high-level language is equivalent to several machine-code instructions, therefore it is more compact than equivalent low-level programs.

- High-level languages are used to solve problems and are often described as **problem-oriented languages**

High-level language (Contd…):

Examples of HLL:

- BASIC was designed to be easily learnt by first-time programmers;
- COBOL is used to write programs solving business problems;
- FORTRAN is used for programs solving scientific and mathematical problems.
- With the increasing popularity of windows-based systems, the next generation of programming languages was designed to facilitate the development of GUI interfaces;

  for example, Visual Basic wraps the BASIC language in a graphical programming environment.
- Support for object-oriented programming has also become more common, for example in C++ and Java.

Example (C program to add 2 numbers):

```
#include<stdio.h>        //header files
Void main()
{
 int a, b, c;            // declaration of 3 variables
 printf("Enter two numbers:\n");
 Scanf("%d", &a);       // read 1st number
 Scanf("%d", &b);      // read 2nd number
 c=a+b;                // compute the sum
 printf("Sum of 2 numbers is %d", c); //print sum
}
```

The C Language

- Currently, the most commonly-used language for embedded systems
- “High-level assembly”
- Very portable: compilers exist for virtually every processor
- Easy-to-understand compilation
- Produces efficient code
- Fairly concise

C History



- Developed between 1969 and 1973 alo
  Unix
- Due mostly to Dennis Ritchie
- Designed for systems programming
  - Operating systems
  - Utility programs
  - Compilers
  - Filters
- Evolved from B, which evolved from BCPL

- **Designed for systems programming**
  - Operating systems
  - Utility programs
  - Compilers
  - Filters
- The current C programming language standard (C11) ISO/IEC 9899 was adopted by ISO and IEC in 2011.

The C Standard Library

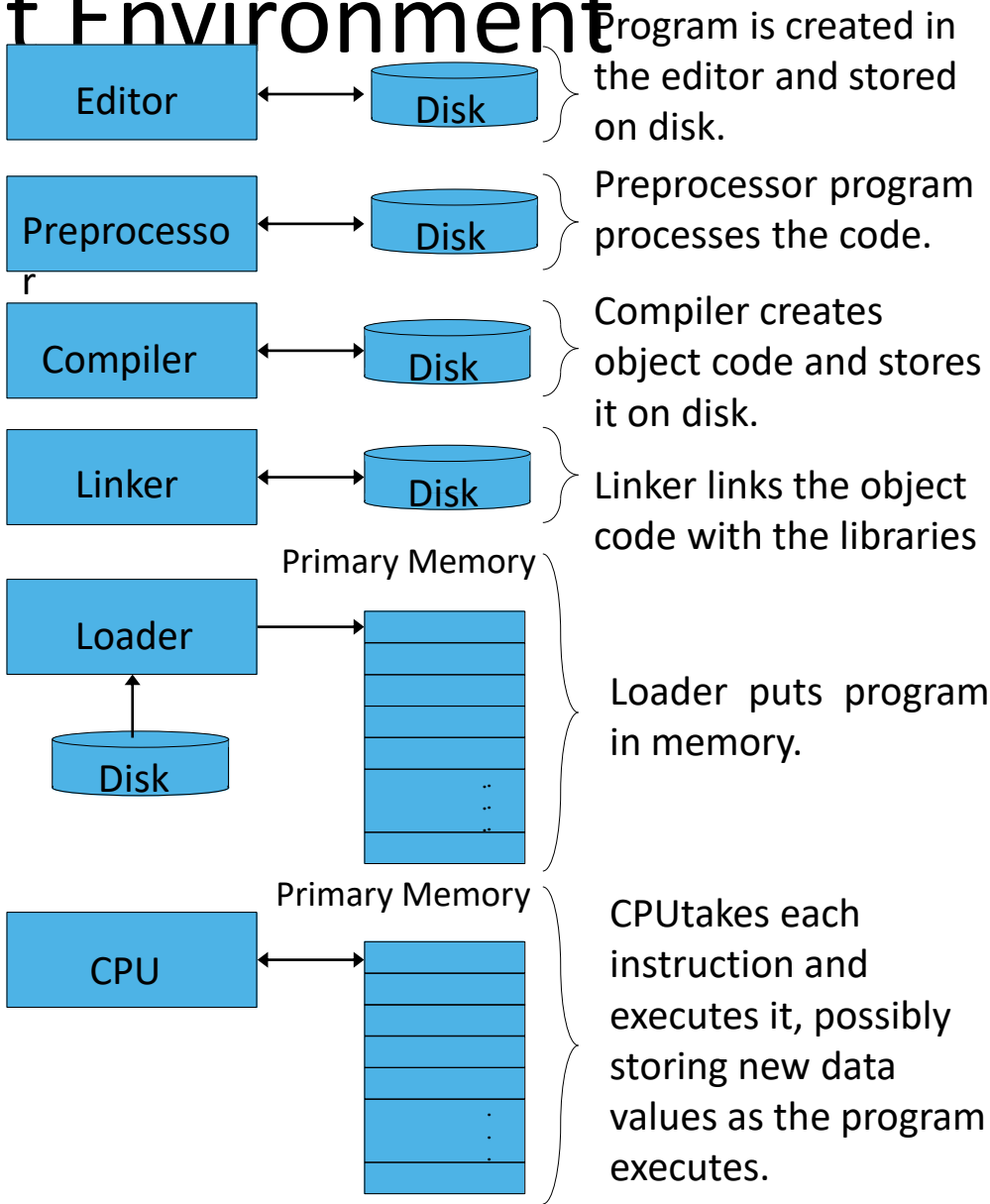
- C programs consist of pieces/modules called functions
  - A programmer can create his own functions
    - Advantage: the programmer knows exactly how it works
    - Disadvantage: time consuming
  - Programmers will often use the C library functions
    - Use these as building blocks
  - Avoid re-inventing the wheel
    - If a premade function exists, generally best to use it rather than write your own
    - Library functions carefully written, efficient, and portable

- **Dev-C++**
- **A free, portable, fast and simple C/C++ IDE**
- a great development environment for beginners
- Uses GCC as its compiler
- Syntax highlighting
- Code completion
- debugging
    - https://sourceforge.net/projects/orwelldevcpp/

- Visual Studio. Eclipse, Netbeans, Code Block, Code lite etc.

Basics of a Typical C Program Development Environment

- Phases of C Programs:
  1. *Edit*
  2. *Preprocess*
  3. *Compile*
  4. *Link*
  5. *Load*
  6. *Execute*



Writing C Programs

- A programmer uses a **text editor** to create or modify files containing C code.
- Code is also known as **source code**.
- A file containing source code is called a **source file**.
- After a C source file has been created, the programmer must **invoke the C compiler** before the program can be **executed** (**run**).

Stages of Compilation

Stage 1: **Preprocessing**

- Performed by a program called the **preprocessor**
- Modifies the source code (in RAM) according to **preprocessor directives (preprocessor commands**) embedded in the source code
- Strips comments and white space from the code
- The source code as stored on disk is <u>not</u> modified.

Stages of Compilation (con't)

Stage 2: **Compilation**

- Performed by a program called the **compiler**
- Translates the preprocessor-modified source code into **object code (machine code)**
- Checks for **syntax errors** and **warnings**
- Saves the object code to a disk file, if instructed to do so.
  - If any compiler errors are received, no object code file will be generated.
  - An object code file will be generated if only warnings, not errors, are received.

Stages of Compilation (con't)

Stage 3: **Linking**

- Combines the program object code with other object code to produce the executable file.
- The other object code can come from the **Run-Time Library**, other libraries, or object files that you have created.
- Saves the executable code to a disk file. On the Linux system, that file is called **a.out**.
  - If any linker errors are received, no executable file will be generated.

Anatomy of a C Program

- *program header comment*

- *preprocessor directives (if any)*

- int main (void)
- {
- *statement(s)*
- return 0 ;
- }

Program Header Comment

- A **comment** is descriptive text used to help a reader of the program understand its content.
- All comments must begin with the characters /* and end with the characters */
- These are called **comment delimiters**
- The program header comment always comes first.

Preprocessor Directives

- Lines that begin with a # in column 1 are called **preprocessor directives** (**commands**).
- Example: the **#include <stdio.h>** directive causes the preprocessor to include a copy of the standard input/output header file **stdio.h** at this point in the code.
- This header file was included because it contains information about the printf ( ) function that is used in this program.

stdio.h

- When we write our programs, there are libraries of functions to help us so that we do not have to write the same code over and over.
- Some of the functions are very complex and long. Not having to write them ourselves make it easier and faster to write programs.
- Using the functions will also make it easier to learn to program!

int main (void)

- Every program must have a **function** called **main**. This is where program execution begins.
- main() is placed in the source code file as the first function for readability. There must be a function with this name or gcc can not successful compile and link your program.
- The **reserved word** “int” indicates that main() **returns** an integer value.
- The parentheses following the reserved word “main” indicate that it is a function.
- The reserved word “void” means nothing is there.

The Function Body

- A left brace (curly bracket) -- **{** -- begins the **body** of every function. A corresponding right brace -- **}** -- ends the function body.

- The style is to place these braces on separate lines in column 1 and to indent the entire function body 3 to 5 spaces.

printf (“Hello, World!\n”) ;

- This line is a C **statement**.
- It is a **call** to the function **printf ( )** with a single **argument (parameter)**, namely the **string** “Hello, World!\n”.
- Even though a string may contain many characters, the string itself should be thought of as a single quantity.
- Notice that this line ends with a semicolon. All statements in C end with a semicolon.

return 0 ;

- Because function main() returns an integer value, there must be a statement that indicates what this value is.
- The statement

  return 0 ;

  indicates that main() returns a value of zero to the operating system.
- A value of 0 indicates that the program successfully terminated execution.
- Do not worry about this concept now. Just remember to use the statement.

Tokens

- The smallest element in the C language is the token.
- It may be a single character or a sequence of characters to form a single item.

Tokens are:

- Tokens can be:
  - Numeric constants
  - Character constants
  - String constants
  - Keywords
  - Names (identifiers)
  - Punctuation
  - Operators

Numeric Constants

- Numeric constants are an uninterrupted sequence of digits (and may contain a period). They never contain a comma.
- Examples:
  - 123
  - 98.6
  - 1000000

Character Constants

- Singular!
- One character defined character set.
- Surrounded on the single quotation mark.
- Examples:
  - ‘A’
  - ‘a’
  - ‘$’
  - ‘4’

String Constants

- A sequence characters surrounded by double quotation marks.
- Considered a single item.
- Examples:
  - “UMBC”
  - “I like ice cream.”
  - “123”
  - “CAR”
  - “car”

Keywords

- Sometimes called reserved words.
- Are defined as a part of the C language.
- Can not be used for anything else!
- Examples:
  - int
  - while
  - for

Names

- Sometimes called identifiers or labels.
- Can be of anything length, but on the first 31 are significant (too long is as bad as too short).
- Are case sensitive:
  - abc is different from ABC
- Must begin with a letter and the rest can be letters, digits, and underscores.
- Must follow the standards for this course!

Punctuation

- Semicolons, colons, commas, apostrophes, quotation marks, braces, brackets, and parentheses.
- ; : , ‘ “ [ ] { } ( )

Operators

- There are operators for:
  - assignments
  - mathematical operations
  - relational operations
  - Boolean operations
  - bitwise operations
  - shifting values
  - calling functions
  - subscripting
  - obtaining the size of an object
  - obtaining the address of an object
  - referencing an object through its address
  - choosing between alternate subexpressions